

Carbon Ready — dMRV Working Document

Solar Rooftop Digital MRV Platform · Sprint 1 + Sprint 2

เอกสารฉบับนี้อธิบายการทำงานของทั้งระบบ (ภาษาไทย + English). หัวข้อแต่ละส่วนจะมีสรุปภาษาไทยก่อน แล้วตามด้วยรายละเอียดภาษาอังกฤษ. This document explains how the whole platform works. Each section leads with a Thai summary, then English detail.

App	Carbon Ready (carbon-ready/)
Domain	Digital MRV (Monitoring, Reporting, Verification) สำหรับคาร์บอนเครดิตจากโซลาร์รูฟท็อป
Branch	feat/sprint-1-mvp
Status	Sprint 1 (MVP) + Sprint 2 (Evidence · Verification · Advanced Audit) merged
Next	Sprint 3 — Hedera Guardian anchoring (เตรียม architecture ไว้แล้ว)

1. ภาพรวม / Overview

ไทย: Carbon Ready เป็นแพลตฟอร์มที่ช่วยให้เจ้าของโครงการโซลาร์บันทึกข้อมูลการผลิตไฟฟ้า คำนวณคาร์บอนที่ลดได้ รวบรวมหลักฐาน (evidence) และส่งให้ผู้ตรวจสอบ (verifier) อนุมัติ — โดยทุกการเปลี่ยนแปลงถูกบันทึกใน audit log แบบแก้ไขไม่ได้ และ "ผูกแอช" ต่อกันเป็นลูกโซ่เพื่อป้องกันการปลอมแปลง. เมื่ออนุมัติแล้ว ระบบจะ "ล็อก" แพ็กเกจและสร้างค่าแฮชไว้ พร้อมให้ Sprint 3 นำไป anchor บน Hedera Guardian.

EN: Carbon Ready digitizes the full MRV loop for rooftop solar carbon credits: capture generation data → compute reductions → collect evidence → submit for third-party verification → approve & lock. Every state change is written to an append-only, hash-chained audit log. Approved packages are frozen and hashed, ready to be anchored on Hedera Guardian in Sprint 3.

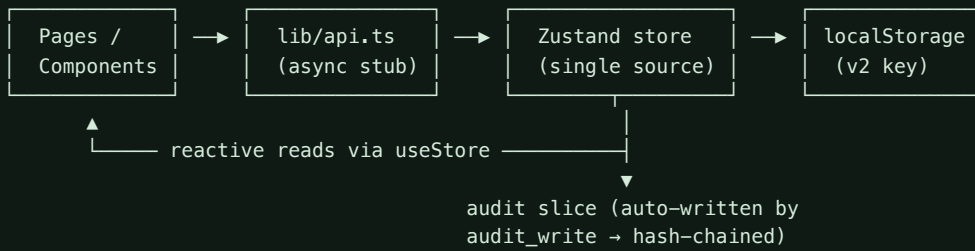
The MRV loop:

```
Monitoring → Calculation → Evidence → Verification → Approved & hash-sealed
(CSV · S1)      (S1)          (S2)          (S2)          (→ Sprint 3 anchor)
```

2. สถาปัตยกรรม / Architecture

ไทย: เป็น React SPA ฟัง frontend ล้วน (ยังไม่มี backend จริง). ข้อมูลทั้งหมดเก็บใน Zustand store + persist ลง localStorage. ทุกหน้าจ้ออ่าน/เขียนผ่าน store และ lib/api.ts (ซึ่งเป็น stub จำลอง async). Sprint 3 จะแทน lib/api.ts ด้วย fetch() จริงโดยไม่ต้องแก้หน้า UI.

EN: Frontend-only React SPA. State lives in a Zustand store persisted to localStorage. UI reads/writes through the store and a thin lib/api.ts facade that simulates async. The store shape mirrors the documented Postgres schema; api.ts mirrors the REST contracts — so a real backend can be dropped in without touching pages or components.



Key principle / หลักการสำคัญ: ทุก action ที่เปลี่ยน state จะเรียก `audit_write(...)` อัตโนมัติ → ได้ audit log ครอบคลุมเหตุการณ์โดยไม่ต้องเขียนซ้ำในแต่ละที่.

3. Tech Stack

Vite 5 · React 18 · TypeScript 5 (strict) · Tailwind 3 · React Router 6 · Recharts 2 · **Zustand 4 (persist)** · PapaParse 5 · date-fns 3 · Lucide React · Vitest 1.

พอนด์/สี: Inter + emerald brand + slate ink (โหนดเดียวกันทั้ง Sprint 1 และ 2).

4. 5วิธีรับ / How to Run

```
cd carbon-ready
npm install
npm run dev      # http://localhost:5173 (หรือ 5174 ถ้าพอร์ตชน)
npm test         # vitest - 25 tests
npm run build    # tsc -b + vite build (production)
npx tsc -b       # type-check only (ไม่มี npm script แยก ใช้คำสั่งนี้)
```

Reset / ล้างข้อมูลกลับเป็น seed: เปิด DevTools console แล้วรัน

```
localStorage.removeItem('carbon-ready-store-v2'); location.reload();
```

หมายเหตุ: Sprint 2 เปลี่ยน persist key จาก `...-v1` เป็น `...-v2` เพื่อให้ seed ของ evidence/verification โคลดใหม่.

5. โครงสร้างโปรเจกต์ / Project Structure

```
carbon-ready/src/
├── App.tsx                # routes
├── layouts/AppShell.tsx   # sidebar + topbar + <Outlet/>
├── pages/
│   ├── Dashboard.tsx     # [S1] KPIs + charts
│   ├── Projects.tsx      # [S1] list / create / edit
│   ├── ProjectDetail.tsx # [S1+S2] tabs: Monitoring | Evidence
│   ├── Upload.tsx        # [S1] CSV upload + validation
│   ├── Calculations.tsx  # [S1] carbon calc
│   ├── EmissionFactors.tsx # [S1] EF master data + versioning
│   ├── Verifications.tsx # [S2] global review queue * new
│   ├── ReviewDetail.tsx  # [S2] /verifications/:id * new
│   └── AuditLog.tsx      # [S1+S2] hash-chain verify + diffs
├── components/
│   ├── Button/Card/Badge/Modal/Drawer/Table/Input/Select/Textarea/... # [S1] shared UI
│   ├── KpiCard/PageHeader/EmptyState/FileDrop/Sidebar/TopBar           # [S1]
│   ├── StatusBadge.tsx          # [S2] StatusBadge + CategoryChip + FileKindIcon *
│   ├── ProjectEvidenceTab.tsx   # [S2] evidence library (project-scoped) *
│   ├── EvidenceUploadModal.tsx # [S2] drag-drop upload *
│   └── EvidenceDetailDrawer.tsx # [S2] detail + version history *
├── store/
│   ├── index.ts               # zustand store + all actions
│   ├── audit.ts               # newAudit() + auditRowHash() (hash chain)
│   └── selectors.ts           # dashboard aggregation
├── lib/
│   ├── api.ts                 # async facade (stub for backend)
│   ├── calc.ts / csv.ts       # [S1] pure fns (tested)
│   ├── date.ts / format.ts    # formatters (+ formatBytes *)
│   ├── geo.ts                 # location → country code
│   ├── labels.ts              # [S2] CATEGORY/STATE/ROLE labels *
│   ├── hash.ts                # [S2] shortHash + canonical JSON *
│   ├── data/seed.ts           # all seed data (incl. S2 evidence/verifications/chained audit)
└── types/index.ts             # all domain types
```

* = ฟีเจอร์/ส่วนที่เพิ่มใน Sprint 2.

6. โมดูล / Modules

Sprint 1 (MVP)

Module	คำอธิบาย (ไทย)	Route
Dashboard	KPI การผลิต/การลดคาร์บอน + กราฟ	/dashboard
Projects	จัดการโครงการ (list/create/edit/detail)	/projects
Upload	อัปโหลด CSV ข้อมูลการผลิต + ตรวจสอบ validation	/upload
Calculations	คำนวณคาร์บอนที่ลดได้ต่อโครงการ	/calculations
Emission Factors	ค่าการปล่อยตามประเทศ/แหล่ง + versioning	/emission-factors
Audit Log	บันทึกทุกการเปลี่ยนแปลง	/audit-log

Sprint 2 (ที่เพิ่มใหม่)

Module	คำอธิบาย (ไทย)	EN	Where
Evidence Management	อัปโหลด/จัดเวอร์ชัน/ค้นหา/archive คลังฐาน ผูกกับแต่ละโครงการ	Document repo per project (PDF/JPG/PNG/XLSX)	Tab ใน /projects/:id
Verification Workflow	คิวรีวิว + state machine + comment + approve/reject/revision	Review queue + lifecycle	/verifications , /verifications/:id
Advanced Audit Trail	audit แบบผูกแฮชต่อกัน + diff ก่อน/หลัง + ตรวจสอบความถูกต้องของลูกโซ่	Tamper-evident hash chain	/audit-log

7. โครงสร้างข้อมูล / Data Model

ไทย: ทุก entity ใช้ snake_case และ id เป็น string มี prefix (prj- , ev- , VR- , aud-). ตารางหลัก:

```
projects —< monitoring_records
          |< calculation_results
          |< evidence_files           (* S2)  parent_id → version chain
          |< verification_requests  (* S2)
              |< evidence_ids[] → evidence_files
              |< verification_comments (* S2)
audit_logs (ผูกแฮชต่อกัน · * extended S2)
users (admin | project_owner | esg_manager | verifier)
```

Sprint 2 types (types/index.ts):

```
EvidenceFile {
  id; project_id; parent_id;           // parent_id ชี้เวอร์ชันก่อนหน้า
  category;                             // 7 หมวด (meter_reading ... verification_report)
  file_name; kind; file_size;
  version_number; status;               // active | superseded | archived
  content_hash; uploaded_by(_name); uploaded_at;
}

VerificationRequest {
  id; project_id; owner_name; assigned_verifier_name;
  state;                               // draft ... approved | rejected
  monitoring_period_start/end; reduction_kgco2e; factors_snapshot;
  evidence_ids[]; required_categories[];
  submitted_at; locked_at; sla_target_days; rejection_reason?;
  hash_value; credential_id; anchored_at; // * Sprint 3 readiness (nullable)
}

VerificationComment { id; verification_id; evidence_id?; author_*; body; reply_to?; created_at; }

AuditLog { // extended in S2
  ...existing...; user_role?; ip_address?;
  previous_value?; new_value?;           // field-level diff
  row_hash?; prev_row_hash?;             // tamper-evident chain
  hcs_topic_id; hcs_sequence_number; // * Hedera (Sprint 3, ยังเป็น null)
}
```

8. Verification Workflow — สถานะและขั้นตอน

ไทย: แพ็กเกจการตรวจสอบไคลตาม state machine นี้ (บังคับใน store actions):

```

draft —submit→ submitted —open→ under_review —approve→ approved (locked, hashed) [OK]
                                     ▲
                                     |
                                     |—resubmit→
                                     |
                                     |—request revision→ revision_required
                                     |
                                     |—reject→ rejected [X]

```

Action (store)	จาก → ไป	บทบาทที่ทำได้
<code>submitVerification</code>	draft → submitted	project_owner
<code>startReview</code>	submitted → under_review	verifier
<code>requestRevision</code>	under_review → revision_required	verifier (ต้องมี summary)
<code>approveVerification</code>	under_review → approved	verifier/approver → ล็อก + เขียน <code>hash_value</code>
<code>rejectVerification</code>	* → rejected	verifier (ต้องมีเหตุผล)
<code>addComment</code>	(ไม่เปลี่ยน state)	ทุกบทบาท

EN: On **approve**, the package is frozen (evidence becomes read-only), `locked_at` + `hash_value` are written, and a `VERIFICATION_APPROVED` audit row is appended to the chain — the exact substrate Sprint 3's anchor worker will consume.

9. Audit Trail & Hash Chain — ระบบกันปลอมแปลง

ไทย: ทุกแถวใน `audit_logs` มี `row_hash` ที่คำนวณจาก แฮชของแถวก่อนหน้า + เนื้อหาแถวนี้ (canonical JSON). ถ้ามีใครแอบแก้ไขแถวไหน ลูกโซ่จะ "ขาด" และตรวจจับได้. หน้า Audit Log จะ **คำนวณลูกโซ่ใหม่ในเบราว์เซอร์** แล้วโชว์แบนเนอร์ "Hash chain verified · N rows".

EN:

```

row_hash = shortHash( prev_row_hash || canonical({
  user_id, user_role, action, entity_type, entity_id,
  previous_value, new_value, ip_address, created_at
}) )

```

- `lib/hash.ts` — `shortHash()` (stand-in for SHA-256) + `canonical()` (key-sorted JSON for stable hashing).
- `store/audit.ts` — `auditRowHash()` + `newAudit()` (used by both the live store and `seed.ts`, so seed and runtime hash identically).
- `pages/AuditLog.tsx` — `verifyChain()` recomputes oldest→newest and compares; renders verified / broken.

⚠ `shortHash` ไม่ใช่ cryptographic hash จริง — เป็นตัวแทนชั่วคราว. Sprint 3 จะเปลี่ยนเป็น SHA-256 จริงตอนต่อ Hedera.

10. State & API Facade — การจัดการข้อมูล

ไทย: หน้า UI อ่าน state แบบ reactive ผ่าน `useStore((s) => ...)` และเรียก action ของ store โดยตรงเมื่อจะเขียน.

`lib/api.ts` มี method async (เช่น `api.uploadEvidence`, `api.approveVerification`) ที่ห่อ action เดิมด้วยดีเลย์จำลอง — มีไว้ให้ Sprint 3 สลับเป็น `fetch()` ได้ทันที.

EN: Reads are reactive via `useStore` ; writes call store actions. `lib/api.ts` wraps those actions in a `tick()` async simulation and is the seam where real HTTP calls land in Sprint 3 — no page/component changes required.

11. วิธีต่อยอด / How to Extend

เพิ่ม evidence category ใหม่: แก้ `EvidenceCategory` ใน `types/index.ts` → เพิ่ม label ใน `lib/labels.ts` (`CATEGORY_LABEL`) → เพิ่ม tone ใน `components/StatusBadge.tsx` (`categoryTone`).

เพิ่ม audit action ใหม่: เพิ่มใน `AuditAction` (`types`) → เพิ่มใน `ACTIONS / TONE` ของ `pages/AuditLog.tsx` → เรียก `audit_write('YOUR_ACTION', entity, id, payload, { previous_value, new_value })` ใน store action.

เพิ่ม state transition: เพิ่ม action ใน `store/index.ts` (อัปเดต state + เรียก `audit_write`) แล้ว expose ผ่าน `lib/api.ts`.

ต่อ backend จริง (Sprint 3): แทน body ของ method ใน `lib/api.ts` ด้วย `fetch()` ที่ยิงไป REST ตาม contract เดิม — store จะกลายเป็น cache/optimistic layer.

12. Sprint 3 — Hedera Guardian Readiness

ไทย: Sprint 2 เตรียม "ตะเข็บ" ไว้ให้ Sprint 3 ต่อ blockchain โดยไม่ต้องแก้ schema:

- `verification_requests.hash_value` — เขียนตอน approve (มีค่าแล้ว)
- `verification_requests.credential_id` / `anchored_at` — ยังเป็น `null`, ให้ anchor worker ของ Sprint 3 เติม
- `audit_logs.hcs_topic_id` / `hcs_sequence_number` — พลาด Hedera Consensus Service (ยัง `null`)
- ลูกโซ่แฮชของ audit — แฮชล่าสุดของแพ็คเกจที่อนุมัติคือสิ่งที่จะ anchor

EN: Sprint 3 adds a worker that reads approved packages, issues a Verifiable Credential via Hedera Guardian, anchors the hash on HCS, then writes back `credential_id` + `anchored_at`. No Hedera SDK or blockchain code exists yet — only the data seams. รายละเอียดดังลึกลงอยู่ใน `/Users/oppabig/dmrv-sprint2/docs/10-hedera-guardian-readiness.md`.

13. Testing

```
npm test      # vitest run — 25 tests (csv.test.ts x13, calc.test.ts x12)
```

ไทย: ตรวจสอบ `lib/csv.ts` และ `lib/calc.ts` (pure functions ที่ใช้ `validate` + คำนวณ). Sprint 2 ไม่ได้ลดความครอบคลุมของเทสเดิม (ผ่านครบ 25). แนะนำเพิ่มเทสสำหรับ state machine ของ `verification` และ `verifyChain()` ใน Sprint ถัดไป.

14. ข้อมูลตัวอย่าง / Sample Data (`data/seed.ts`)

- **Projects:** Pune (IN, 250 kWp) · Bangkok (TH, 820 kWp) · Hanoi (VN, draft)
- **Monitoring:** ~360 records, Dec 2025 → May 2026
- **Emission factors:** IN/CEA v1+v2, TH/EGAT, VN/EVN
- **Evidence (*):** 9 ไฟล์ (Pune + Bangkok) รวมตัวอย่าง version chain (meter v1→v2) และ archived

- **Verifications (*)**: VR-1000 approved/locked · VR-1001 under_review · VR-1002 submitted · VR-1003 revision_required
 - **Audit (*)**: chained log ของเหตุการณ์ Sprint 1 + Sprint 2 (verify ได้จริงในหน้า Audit Log)
- ทดลองใช้: Projects → **Pune Rooftop Phase 1** → tab **Evidence** → Upload (มีปุ่ม "add sample files"). แล้วไป **Verifications** → มีปุ่ม VR-1001 → comment / request revision / approve.

15. อ้างอิง / References

- Sprint 1 spec: ../../docs/superpowers/specs/2026-05-27-carbon-ready-sprint-1-design.md
- Sprint 1 plan: ../../docs/superpowers/plans/2026-05-27-carbon-ready-sprint-1.md
- Sprint 2 planning pack (PRD · user stories · DB · API · UX · security · backlog · AC · Hedera):
/Users/oppabig/dmrv-sprint2/docs/
• App README: ../../README.md

Generated as the Sprint 2 working document. Update this file when modules, schema, or workflow change.